A TECHNICAL SEMINAR REPORT
ON

FASTAPI

Submitted in partial fulfillment of requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

Submitted by:

KAYAMKHANI THASLEEM (22091A32F5)

Under the Supervision of

Supervisor

Dr. M. Suleman Basha, M. Tech, Ph. D.

Associate Professor, Dept. of CSE (DS)

Co-Supervisor

Ms. C. Supraja M. Tech,

Assistant Professor, Dept. of CSE(DS)

(ESTD-1995)

DEPARTMENTOF COMPUTER SCIENCEAND ENGINEERING (DATA SCIENCE)

RAJEEV GANDHI MEMORIAL COLLEGE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)
Affiliated to JNTUA Anantapuramu, APPROVED BY A.I.C.T.E., NEW DELHI, ACCREDITED BY NAAC of UGC, NEWDELHI with "A+" grade, ACCREDITED BY N.B.A, NEWDELHI,
NANDYAL-518501 Dist. A.P

YEAR: 2025-2026

Rajeev Gandhi Memorial College of Engineering &Technology

(AUTONOMOUS)

Approved by AICTE, New Delhi; Affiliated to JNTUA-Ananthapuramu,
Accredited by NBA (6-Times); Accredited by NAAC with 'A+' Grade (Cycle-3), New Delhi;
World Bank Funded Institution; Recognized UGC-DDU Kaushal Kendra;

NANDYAL – 518501 (ESTD -1995)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

(ESTD-1995)

CERTIFICATE

This is to certify that the report entitled "**FASTAPI**" is a bonafide record of the technical seminar done by **KAYAMKHANI THASLEEM (22091A32F5)** in partial fulfillment of the requirements for the award of Degree of Bachelor of Technology in **Computer Science & Engineering (DATA SCIENCE)** from RGM College of Engineering & Technology, Nandyal during the academic year 2025-26.

Supervisor

Dr. M. Suleman Basha, M. Tech, Ph. D.

Associate Professor Dept. of CSE (DS)

Head of the Department

Dr. B. Bhaskara Rao, M. Tech, Ph. D.

Associate Professor & HOD, Dept. of CSE (DS)

Co-Supervisor

Ms. C. Supraja M. Tech,

Assistant Professor, Dept. of CSE(DS)

Place: Nandyal
Date:

Candidate's Declaration

I hereby declare that the technical seminar entitled **"FASTAPI"** was carried out and written by me under the guidance of **Dr. M. Suleman Basha,** Associate Professor and **Ms. C. Supraja,** Assistant Professor, Dept. of CSE(DS), R.G.M. College of Engineering & Technology (AUTONOMOUS), and this technical seminar report is submitted in the partial fulfillment of the requirements for the award of the degree of "Bachelor of Technology" in **Computer Science & Engineering (Data Science).**

(**KAYAMKHANI THASLEEM**)

Place: Nandyal

Roll No.: 22091A3281

Date:

IV B. Tech II Sem, CSE(DS),

RGMCET

ABSTRACT

FastAPI is a modern, high-performance web framework used for building RESTful APIs using Python. It is designed to provide fast development speed along with excellent runtime performance by leveraging Python type hints. FastAPI simplifies API creation by offering automatic request validation, serialization, and error handling, making it highly suitable for building scalable and reliable backend services.

One of the key features of FastAPI is its ability to automatically generate interactive API documentation using OpenAPI standards. This documentation is available through user-friendly interfaces such as Swagger UI, which allows developers to test APIs directly from the browser. FastAPI also supports asynchronous programming, enabling efficient handling of multiple client requests with minimal latency.

Due to its speed, simplicity, and built-in features, FastAPI is widely used in developing microservices, cloud-based applications, and machine learning APIs. It is increasingly preferred over traditional frameworks for modern application development, as it reduces development time while ensuring high performance.

CONTENTS

Chapter No	TITLE	Page no.
	Abstract	
	List of Abbreviations	v
	List of Figures	vi
	List of Tables	vii
1	INTRODUCTION	1
2	HISTORY	2
	2.1 Evolution	2-4
3	WORKING PRINCIPLES	5
	3.1 FastApi Architecture	6
	3.2 Core Components	6
	3.3 FastApi Request Handling Process	7
4	ADVANTAGES	8-9
5	DISADVANTAGES	10-11
6	FUTURE WOR	12-13
	6.1 Scope of Future Application	12
	6.2 Scope of Improvement	13
7	CONCLUSION	14
8	REFERENCES	15
9	APPENDICES	16-17
	A. Sample FastApi Code	16
	B. FastApi Project Structure	17

LIST OF ABBREVIATIONS

API: Application Programming Interface

ASGI: Asynchronous Server Gateway

Interface CRUD: Create, Read, Update,

Delete

CORS: Cross-Origin Resource

Sharing DB: Database

DI: Dependency Injection

FastAPI: Fast Application Programming

Interface HTTP: HyperText Transfer Protocol

HTTPS: HyperText Transfer Protocol Secure

JSON: JavaScript Object Notation

JWT: JSON Web Token

MVC: Model–View–Controller

ORM: Object Relational Mapping

OpenAPI: Open Application Programming Interface Specification

Pydantic: Python Data Validation Library

REST: Representational State

Transfer SQL: Structured Query

Language SQLite: Structured Query

Language Lite

Swagger UI: API Documentation

Interface Uvicorn: ASGI Server

Implementation WSGI: Web Server

Gateway Interface

LIST OF FIGURES

FIGNO.	FIGURE NAME	PAGENO
Fig. 1.1	FastAPI Architecture	3
Fig. 1.2	FastAPI request–response	4

CHAPTER 1

INTRODUCTION

FastAPI represents a modern and efficient approach to building web APIs using Python. It is a high- performance web framework designed for developing RESTful APIs quickly, reliably, and with minimal boilerplate code. Built on top of Starlette for web handling and Pydantic for data validation, FastAPI leverages Python type hints to provide automatic request validation, serialization, and interactive API documentation. Since its release, FastAPI has gained significant popularity among developers for its speed, simplicity, and robustness.

Unlike traditional Python web frameworks that rely heavily on synchronous execution models, FastAPI is based on the ASGI (Asynchronous Server Gateway Interface) standard. This allows it to handle asynchronous requests efficiently, making it well-suited for high-concurrency applications such as microservices, real-time systems, and cloud-native APIs. FastAPI achieves performance comparable to frameworks written in lower-level languages, while retaining the readability and flexibility of Python.

A distinguishing feature of FastAPI is its automatic generation of interactive API documentation using OpenAPI specifications, Swagger UI, and ReDoc. This capability enables developers and testers to explore and test APIs directly from the browser without additional tools. By enforcing strict data validation through Pydantic models, FastAPI reduces runtime errors and improves application reliability, ensuring that only valid data is processed by the system.

FastAPI also provides built-in support for dependency injection, authentication mechanisms, and security standards such as OAuth2 and JSON Web Tokens (JWT). These features make it suitable for building secure and scalable enterprise-grade applications. Its modular design allows seamless integration with databases, machine learning models, and third-party services, making it an ideal framework for modern backend development.

The significance of FastAPI extends beyond simple API creation. It is widely used in developing microservices architectures, deploying machine learning models, and building scalable backend systems for web and mobile applications. With its emphasis on performance, developer productivity, and maintainability, FastAPI plays a crucial role in contemporary software engineering practices.

CHAPTER 2 HISTORY

The history of FastAPI is closely associated with the evolution of Python-based web frameworks and the growing demand for high-performance, developer-friendly API development tools. FastAPI was created by Sebastián Ramírez and was first released in 2018. The framework emerged as a response to limitations observed in existing Python frameworks such as Flask and Django, particularly in terms of performance, built-in data validation, and native support for asynchronous programming

1) Background: Before FastAPI (Pre-2018):

Before FastAPI, Python developers mainly relied on:

- **Flask** – simple but lacked built-in validation and async support
- **Django** – powerful but heavy for API-only services
- **Problems developers faced:**
- Manual data validation
- No automatic API documentation
- Slower performance for async workloads
- Repetitive boilerplate code
- At the same time, modern applications demanded: •
- High performance
- Clean API contracts
- Strong typing

2) Creation of FastAPI (2018):

FastAPI was created in **2018** by **Sebastián Ramírez**, a software engineer from Colombia.

Main goals:

- Use **Python type hints** for validation
- Provide **automatic documentation**
- Support **async/await** natively
- Achieve **top-tier performance**
- Sebastián previously experimented with multiple frameworks and realized none fully satisfied modern API requirements.

3) Technical Foundations:

FastAPI is built by combining powerful existing tools:

- Built on **Starlette**
- Enables:
  - Async request handling
  - WebSockets
  - Background tasks
  - Extremely fast performance

Data Validation

- Uses **Pydantic**
- Converts Python type hints into:
  - Runtime validation
  - Automatic serialization
  - Clear error messages

API Standards

- Automatic OpenAPI (Swagger) compliance
- Interactive docs generated without extra code

4) Early Adoption Phase (2019):

FastAPI gained attention because it:

- Eliminated repetitive validation code
- Reduced bugs using type safety
- Generated docs

automatically Developers

appreciated:

- Editor autocomplete
- Clear request/response models
- Minimal configuration

GitHub stars grew rapidly during this period.

5) Explosion in Popularity (2020):

In **2020**, FastAPI became widely known due to:

- Rise of **microservices**
- Growth of **machine learning APIs**
- Increased need for **cloud-native**

backends Performance

- Benchmarks showed FastAPI performance comparable to:
  - Node.js

6) Enterprise & Industry Adoption(2021–2023):

Major companies started using FastAPI in production:

- Netflix
- Uber
- Microsoft

Why enterprises adopted it:

- Scalable architecture
- Easy integration with Kubernetes, Docker
- Excellent API documentation for teams

FastAPI became a **default choice** for:

- REST APIs
- AI/ML model serving

7) Current State (2024–Present):

Today, FastAPI is:

- Actively maintained
- Community-driven
- Considered an **industry standard** for APIs

CHAPTER 3

WORKING PRINCIPLES

3.1 FastAPI Architecture:

At its core, FastAPI is built on a modern ASGI (Asynchronous Server Gateway Interface)–based architecture, which enables high-performance, asynchronous request handling. Unlike traditional WSGI-based frameworks, FastAPI supports non-blocking I/O operations, allowing it to efficiently manage multiple concurrent client requests. This architectural design makes FastAPI particularly suitable for scalable web services and microservice-based systems.

FastAPI internally relies on Starlette for routing, middleware support, and asynchronous networking. Incoming HTTP requests are processed through a routing layer that maps endpoints to corresponding path operation functions. These functions can be synchronous or asynchronous, providing developers with flexibility based on application requirements. The framework efficiently handles request parsing, execution, and response generation with minimal overhead.

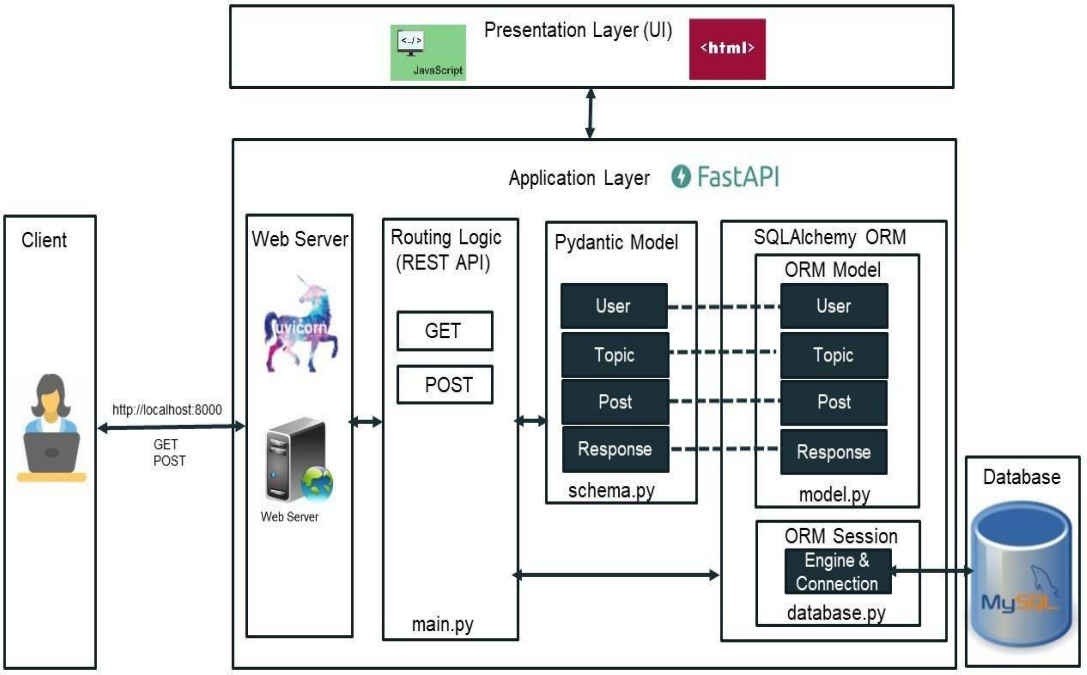

Fig 1.1: FastAPI Architecture

Another significant aspect of FastAPI's architecture is its tight integration with Pydantic for data validation and serialization. Input data received through path parameters, query parameters, headers, or request bodies automatically validated against predefined data

models. This ensures data integrity and reduces runtime errors, while also simplifying backend logic.

FastAPI also automatically generates OpenAPI-compliant specifications, which serve as the foundation for interactive API documentation using Swagger UI and ReDoc. This architectural feature enhances transparency and usability by enabling developers and consumers to understand and test APIs without external documentation tools.

3.2 Core Functional Principles of FastAPI :

One of the defining principles of FastAPI is the use of Python type hints as the central mechanism for defining API behavior. Instead of relying on extensive configuration or manual validation logic, FastAPI interprets type annotations to perform request validation, response serialization, and error handling automatically. This approach improves code readability and maintainability while reducing development time.

FastAPI incorporates a powerful dependency injection system, which allows reusable components such as database connections, authentication handlers, and configuration settings to be injected into path operations. This promotes modular design and enhances testability by decoupling application components.

FastAPI incorporates a powerful dependency injection system, which allows reusable components such as database connections, authentication handlers, and configuration settings to be injected into path operations. This promotes modular design and enhances testability by decoupling application components.

Security is another fundamental working principle of FastAPI. The framework provides built-in support for modern authentication and authorization standards, including OAuth2, JWT, and API key-based security.

3.3 FastAPI Request-Response Processing Flow:

When a client such as a browser, mobile app, or API tool sends an HTTP request to a FastAPI application, the request is first received by an ASGI server like **Uvicorn**,.

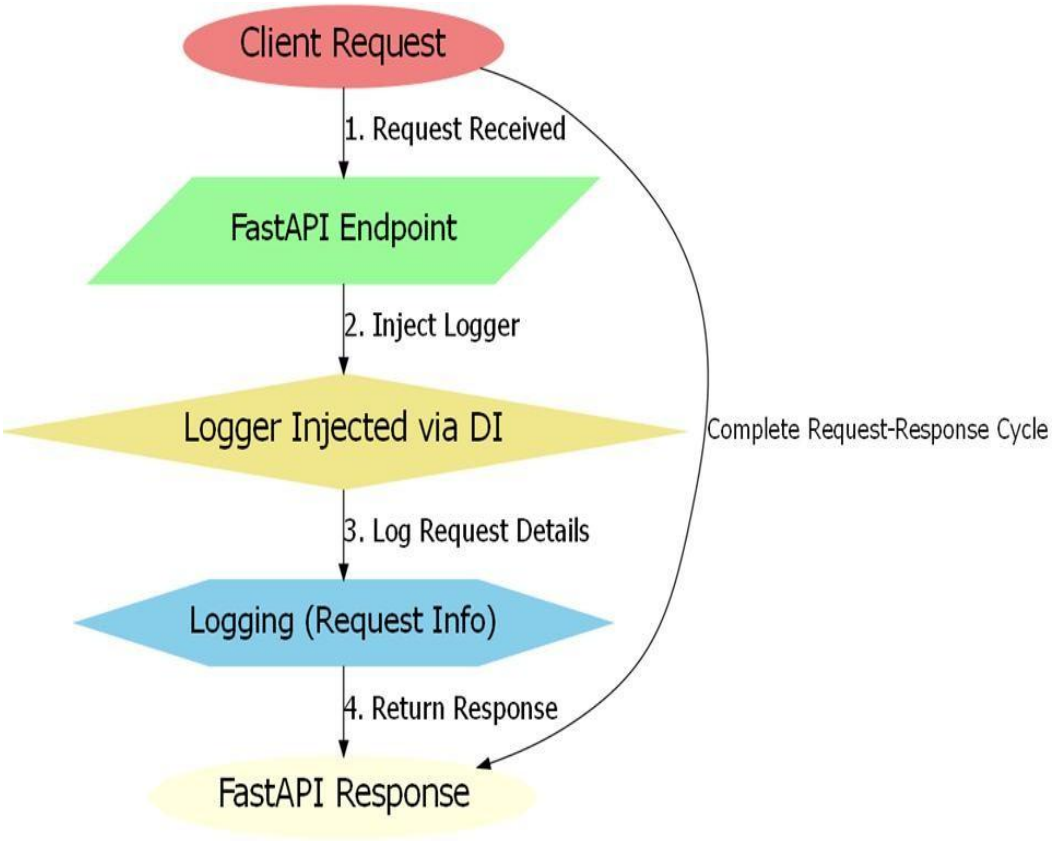

Fig: FastAPI request–response

Before reaching the endpoint, the request passes through middleware provided by **Starlette**, where tasks such as authentication, CORS handling, and logging can be performed. FastAPI then matches the request URL and HTTP method with the appropriate route and resolves all required dependencies using its built-in dependency injection system. After that, the request data—including path parameters, query parameters, headers, and body—is automatically validated and parsed using **Pydantic** based on Python type hints; if validation fails, an error response is immediately returned. Once validation succeeds, the endpoint function is executed to perform the required business logic. The result produced by the endpoint is then serialized into a proper HTTP response, usually JSON, with correct status codes and headers. Finally, the response passes back through the middleware stack for any post-processing and is sent by the ASGI server back to the client, completing the FastAPI request–response lifecycle efficiently and asynchronously.

CHAPTER 4

ADVANTAGES

- **High Performance:**

  FastAPI is built on the ASGI standard and leverages asynchronous programming, enabling it to handle thousands of concurrent requests efficiently. Its performance is comparable to frameworks developed in lower-level languages such as Node.js and Go, making it suitable for high-traffic and real-time applications.

- **Automatic Date Validation:**

  FastAPI uses Python type hints along with Pydantic models to automatically validate incoming request data. This ensures correctness of inputs, reduces runtime errors, and improves overall application reliability without requiring additional validation code.

- **Automatic API Documentation:**

  One of the major advantages of FastAPI is the automatic generation of interactive API documentation using OpenAPI standards. Tools such as Swagger UI and ReDoc are integrated by default, allowing developers and users to test APIs directly through a browser interface.

- **Developer Productivity:**

  FastAPI significantly reduces boilerplate code by using type annotations to define request parameters, response models, and validation rules. This improves code readability, simplifies maintenance, and accelerates development time.

- **Asynchronous and Synchronous Support:**

  FastAPI allows developers to write both synchronous and asynchronous endpoints within the same application. This flexibility enables efficient handling of I/O-bound and CPU-bound operations based on application needs.

- **Powerful Dependency Injection:**

  FastAPI provides a built-in dependency injection system that promotes modular design and code reusability. Components such as database connections, authentication handlers, and configuration settings can be efficiently managed and injected where required.

- **Security and Authentication Support:**

  FastAPI includes built-in support for modern security standards such as OAuth2, JWT, API keys, and HTTP authentication. These features make it suitable for building secure enterprise-grade APIs.

- **Scalability and Microservices Friendly:**
  FastAPI is well suited for microservices architectures due to its lightweight design and high performance. It integrates easily with cloud platforms, containers, and orchestration tools such as Docker and Kubernetes.
- **Strong Community and Documentation:**
  FastAPI has an active open-source community and comprehensive official documentation. This makes it easy for developers to learn, troubleshoot issues, and adopt best practices.
- **Easy Integration with Machine Learning and Data Science Tools:**
  FastAPI integrates seamlessly with popular Python data science and machine learning libraries such as TensorFlow, PyTorch, and Scikit-learn. This makes it an excellent choice for deploying machine learning models as APIs, enabling real-time predictions and data-driven services with minimal overhead.

CHAPTER 5

DISADVANTAGES

- **Steeper Learning Curve for Beginners:**

  FastAPI relies heavily on Python type hints, asynchronous programming, and modern concepts such as dependency injection. Beginners who are new to these concepts may find FastAPI more difficult to learn compared to simpler frameworks like Flask.

- **Limited Built-in Features**:

  Unlike full-stack frameworks such as Django, FastAPI does not provide built-in components like an admin panel, authentication system, or ORM. Developers must rely on third-party libraries to implement these features.

- **Smaller Ecosystem Compared to Django:**

  Although FastAPI is growing rapidly, its ecosystem is still smaller than that of Django. Fewer plugins, extensions, and community-maintained packages are available, which may increase development effort for complex projects.

- **Asynchronous Complexity:**

  Improper use of asynchronous and synchronous code together can lead to performance issues or unexpected behavior. Developers must have a good understanding of async/await concepts to fully utilize FastAPI's performance benefits.

- **Database Integration Requires Extra Setup:**

  FastAPI does not enforce a specific database or ORM. While this provides flexibility, it also requires additional configuration and decision-making for database integration, which can slow down development.

- **Not Ideal for Traditional Server-Rendered Applications:**

  FastAPI is optimized for building APIs rather than server-rendered web pages. Applications that require extensive HTML templating and server-side rendering may be better suited to frameworks like Django.

- **Rapid Updates Can Cause Compatibility Issues:**
  FastAPI and its underlying dependencies evolve quickly. Frequent updates may introduce breaking changes or compatibility issues, requiring developers to regularly maintain and update their applications.
- **Limited Built-in Error Handling Customization:**
  Although FastAPI provides automatic error responses, advanced customization of error handling and logging may require additional middleware or third-party tools, increasing application complexity.

CHAPTER 7

FUTURE SCOPE

The future of FastAPI points toward a growing role in modern software development, where scalable, high-performance, and cloud-native APIs form the backbone of digital infrastructure. As applications transition from monolithic systems to distributed architectures, FastAPI is expected to evolve as a core framework supporting this transformation.

a. Scope of Future Applications:

i. Microservices and Cloud-Native Systems:

FastAPI is expected to be widely adopted in microservices-based architectures due to its lightweight design and high performance. Its compatibility with containerization technologies such as Docker and orchestration platforms like Kubernetes makes it ideal for cloud-native deployments.

ii. Machine Learning Model Deployment:

FastAPI will continue to play a significant role in serving machine learning and deep learning models as RESTful APIs. Its seamless integration with Python-based data science libraries makes it suitable for real-time prediction systems in healthcare, finance, and recommendation engines.

iii. Enterprise Backend Development:

FastAPI can evolve into a standard backend framework for enterprise applications, integrating with ERP, CRM, and HRMS systems. Its security features and scalability support large-scale business operations.

iv. Real-Time and High-Concurrency Applications:

With native asynchronous support, FastAPI is well suited for real-time applications such as chat systems, streaming services, and IoT data pipelines, where handling a large number of concurrent connections is essential.

b. Scope of Improvement:

i. Enhanced Developer Tooling:

Future improvements may include more advanced debugging tools, better error visualization, and enhanced development-time feedback to improve developer experience.

ii. Expanded Plugin and Extension Ecosystem:

FastAPI's ecosystem is still evolving. Future growth is expected in the form of standardized plugins for authentication, logging, monitoring, and database management.

iii. Improved Asynchronous Abstractions:

Simplifying asynchronous programming patterns and reducing complexity for beginners would improve FastAPI's accessibility and adoption.

iv. Better ORM and Database Integrations:

Stronger integration with popular ORMs and database tools could reduce setup overhead and make FastAPI more appealing for rapid application development.

CHAPTER 8
CONCLUSION

The evolution of FastAPI represents a significant milestone in modern web application development, particularly in the domain of high-performance and scalable API design. By combining the simplicity of Python with modern software engineering principles such as asynchronous programming, type annotations, and dependency injection, FastAPI has successfully bridged the gap between developer productivity and system performance. Unlike traditional frameworks that often require trade-offs between speed and ease of development, FastAPI delivers both, making it a compelling choice for contemporary backend systems.

One of the most impactful contributions of FastAPI lies in its API-first philosophy. The automatic generation of interactive documentation using OpenAPI and Swagger not only improves developer experience but also enhances collaboration between frontend developers, backend engineers, and third-party integrators. This self-documenting capability reduces ambiguity, enforces strict data validation, and minimizes runtime errors, thereby improving the reliability of software systems. Furthermore, FastAPI's strong reliance on Pydantic for data modeling ensures consistency and robustness across application layers.

FastAPI has also proven itself as a powerful framework for emerging technological domains such as microservices, cloud-native architectures, and machine learning model deployment. Its lightweight nature and native support for asynchronous execution make it highly suitable for handling large volumes of concurrent requests, which is essential in real-time applications, distributed systems, and API gateways. As organizations increasingly migrate toward containerized and serverless infrastructures, FastAPI's compatibility with Docker, Kubernetes, and modern DevOps pipelines positions it as a future-ready solution.

Despite its advantages, FastAPI also highlights the growing importance of modern programming concepts in software engineering. The framework encourages developers to adopt best practices such as clean architecture, strong typing, and explicit dependency management. While this may present a learning curve for beginners, it ultimately results in more maintainable, scalable, and secure applications. In this sense, FastAPI not only serves as a development tool but also as a catalyst for improving software design discipline.

In conclusion, FastAPI is not merely a web framework but a reflection of the evolving needs of modern software systems. Its emphasis on performance, clarity, and scalability makes it an ideal choice for next-generation applications.